TCD3
Test Automation & Continuous Delivery

Continuous Integration/Delivery/Deployment

S. Wagner & J. Karder

Why Test Automation is so Important

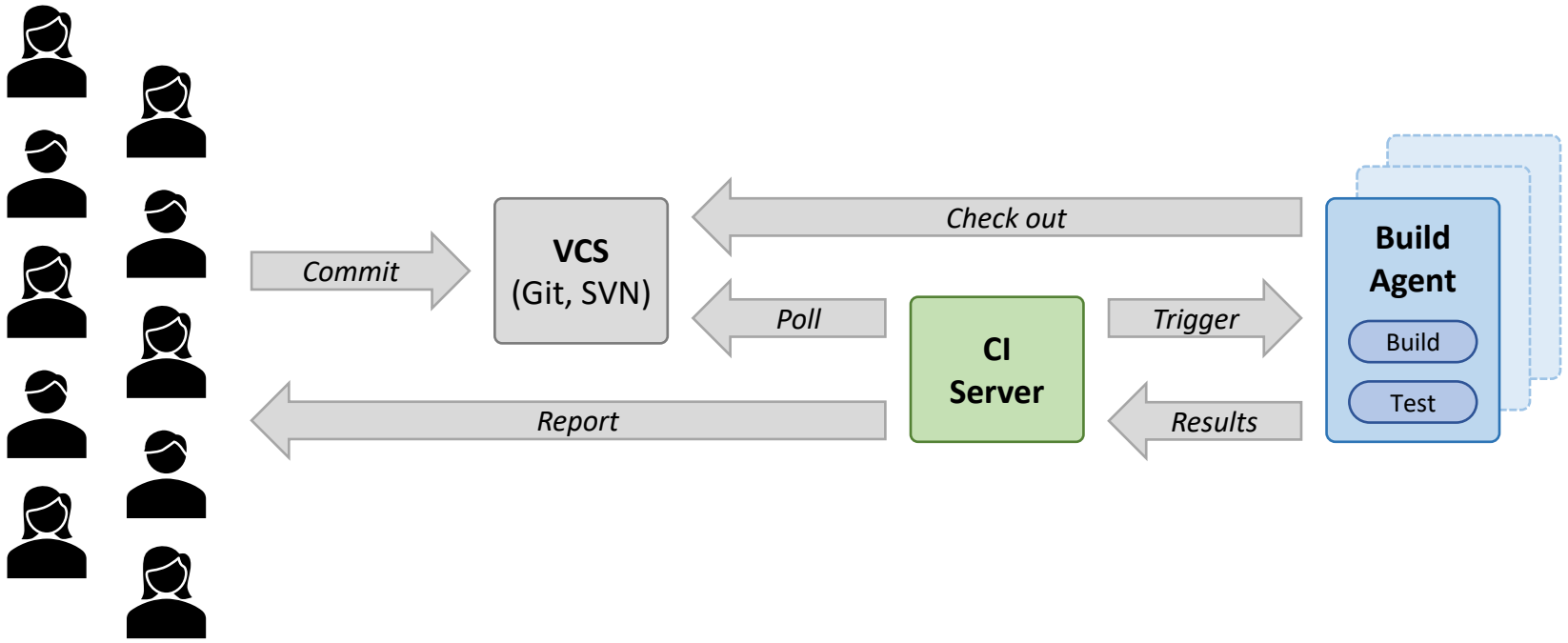
- Benefits of executing tests automatically:
  - you can be sure that the code in the repository does not break anything
  - if something breaks, you get immediate feedback
  - you do not have to execute tests manually and therefore you save time
  - you must keep your test suite up to date and preserve it from rotting
  - you even might get additional feedback (e.g., code coverage, code complexity, adherence to coding standards, performance)
  - you ensure that everything works in a well-defined environment
  - you (eventually) get rid of "It works on my machine."

Continuous Integration (CI)

- Continuous integration automates compilation and executes unit and maybe also integration tests
  - helps to identify problems early (e.g., build breaks, failing tests)
  - CI without automated tests is like a car without wheels

- Prerequisites of CI:
  - version control system (either centralized or decentralized; mostly Git in these days)
  - CI server (aka. build server)
  - optionally some client machines to compile and execute tests on them
  - executable description of the build and test process
  - channel to provide feedback (e.g., e-mail)
  - frequent integration of changes (i.e., commits/pushes to the VCS)

Continuous Integration (CI)



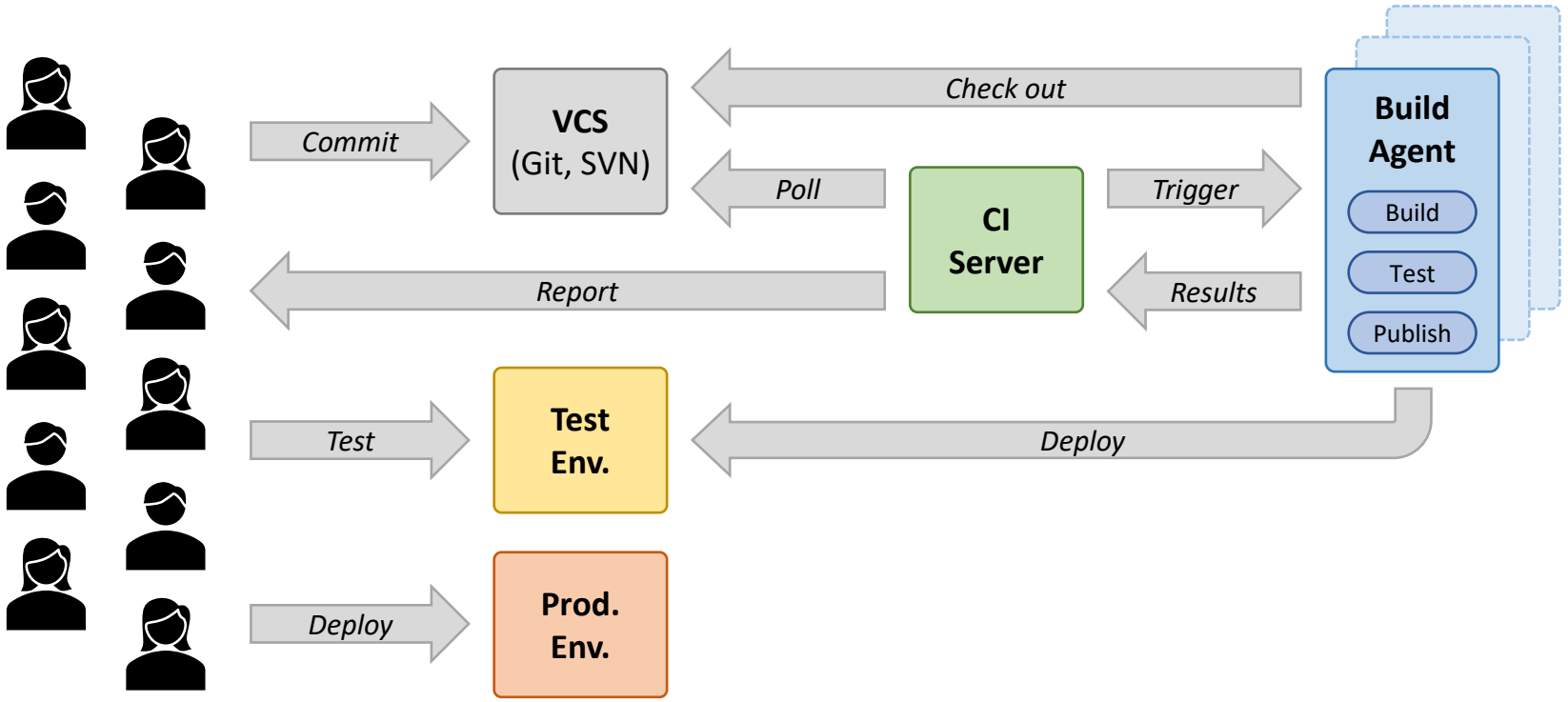
CI Pipelines

- Definition of a CI pipeline
  - consists of all steps which should be executed to create a build and execute tests
  - are executed regularly after each commit/push
  - structured in stages, at least *build* and *test*
  - CI pipelines are often extended by additional steps to check code quality (e.g., static code analyzers)
  - often build agents (i.e., dedicated machines) compile software and execute tests in order to spare resources on the CI server
- When to trigger your CI pipeline?
  - on demand? nightly? on each change?
  - as often as you can afford it – the more often the better
  - if executing tests takes too long, make your tests faster
  - or use different test groups (e.g., unit tests on each change, integration/performance tests nightly)
- Golden rule: "If your CI pipeline fails, it has to be fixed immediately."

Continuous Delivery (CD)

- Compiling the code and executing the tests is not sufficient
- Software should be tested in an environment which is technically very similar to the production environment
  - often there is a significant difference between production environments (multiple machines, security, load balancing, etc.) and development environments (single machine of a developer)
- Deployed versions can be used by testers, product owners, customers, etc. (cf. principles of agile software: deliver working software)
  - system tests can be done manually
- Continuous delivery extends the CI pipeline by this step
- Deployment to the production environment is also often automated, but still manually triggered

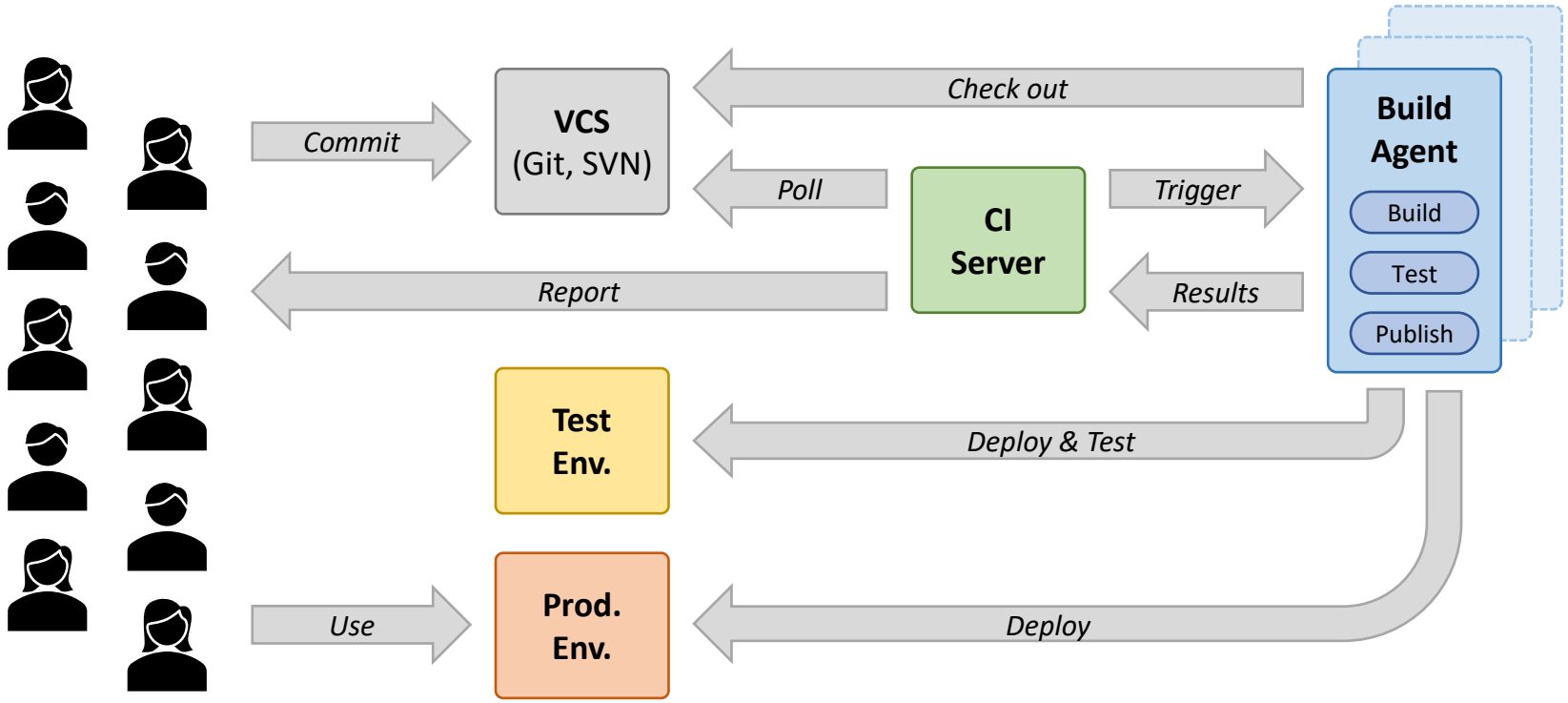
Continuous Delivery (CD)



Continuous Deployment

- System tests in a test environment are also fully automated
- Software is automatically deployed to the production environment, if no errors occur

Continuous Deployment



CI/CD Systems

- Many different tools and vendors
  - open-source, commercial, on premises, SaaS, …
  - some examples: Jenkins, TeamCity, GitLab, GitHub Actions, Azure Pipelines, CircleCI, CloudBees CI, …
- Core functionality is always the same
  - monitor VCS for changes
  - retrieve most recent version
  - prepare environment
  - execute pipeline (compile, test, inspect, deploy)
  - store build artifacts
  - report results

GitLab CI/CD

https://gitlab.com



- GitLab supports CI/CD
  - definition of CI/CD pipeline is stored in the repository (`.gitlab-ci.yml`)
  - each pipeline is executed on a runner
  - runners support different executors (e.g., docker, shell)

- Components of a GitLab CI/CD pipeline:
  - pipelines consist of stages and each stage contains one or more jobs
  - a stage defines when to run jobs, a job defines what to do
  - jobs are isolated from each other
  - all jobs in a stage can run in parallel, if there are enough runners
  - jobs can be restricted to specific runners
  - jobs can be triggered by or excluded from specific branches
  - jobs can also be triggered manually
  - if a single job fails, (usually) the whole pipeline fails

- Online documentation of GitLab's CI/CD features: https://docs.gitlab.com/ee/ci/README.html

GitLab CI/CD

https://gitlab.com



- Types of CI/CD pipelines in GitLab:
  - basic pipelines
    - execute one stage after another and run all jobs in each stage in parallel
  - directed acyclic graph (DAG) pipelines
    - relationships between jobs can be defined as DAG
  - multi-project pipelines
    - a pipeline in one project can trigger pipelines of other projects
  - parent-child pipelines
    - pipelines can be hierarchically structured
  - pipelines for merge requests
    - pipelines can be restricted to commits which belong to a merge request
  - pipelines for merged results
    - pipelines can be executed on the result of a merge request before the merge request is accepted
  - merge trains
    - pipelines for merged results can be queued one after the other

GitLab CI/CD

https://gitlab.com



- Simple example of a `.gitlab-ci.yml` file:

```yaml
stages:                                  # list of all stages
  - build
  - test

image: mcr.microsoft.com/dotnet/sdk:6.0  # docker image, if docker executor is used
                                         # (can also be defined individually in each job)

build_project:                           # id of job
  stage: build
  tags:                                  # list of tags, which a runner must have to execute this job
    - dotnet_runner
  script:                                # list of commands
    - dotnet build

test_project:
  stage: test
  tags:
    - dotnet_runner
  script:
    - dotnet test
```

GitLab CI/CD – Stages

https://gitlab.com



`stages`

- defines the sequence of stages each pipeline is going through
- if missing, defaults to `build`, `test`, `deploy`
- two implicit stages `.pre` and `.post`
  - do not have to be defined
  - can be used to execute jobs before or after all other user-defined stages
  - order of .pre and .post cannot be changed

`tags`

- runners can be tagged with arbitrary labels
- job can be restricted to runners which have all specified tags
- per default, runners do not execute jobs without any tags (can be changed in runner settings)
- tags can be used to mark attributes of runners (e.g, OS, installed frameworks/services, …)

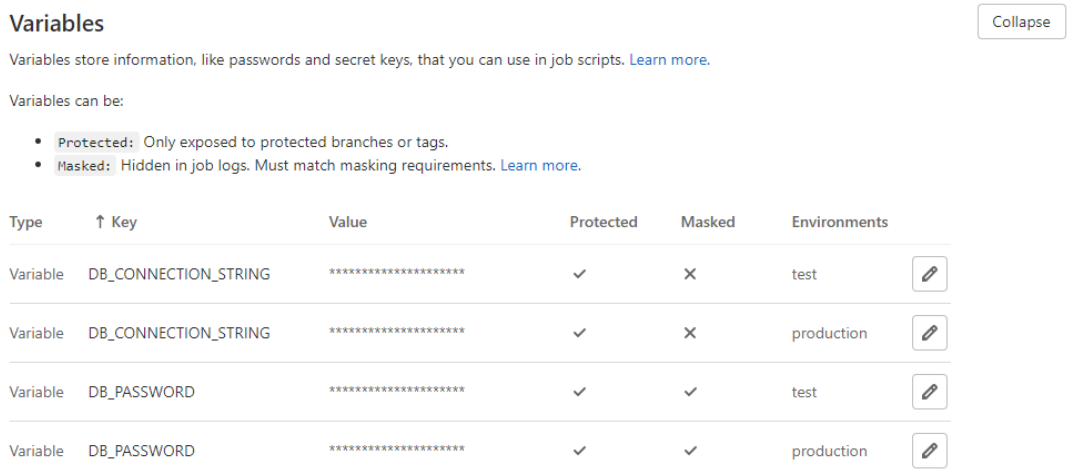
GitLab CI/CD – Variables

https://gitlab.com

`variables`

- can be defined in the pipeline configuration
- can also be defined in the web frontend (Settings → CI/CD → Variables)
  - variables can be protected and/or masked
  - useful for secrets
- values can be used everywhere
- variables are exposed as environment variables
- can be set globally or per job

```yaml
variables:
  nuget_packages_cache: ".nuget"
```



- GitLab also exposes many predefined variables:
  - e.g., `$CI_JOB_NAME`, `$CI_COMMIT_REF_SLUG`, `$CI_COMMIT_SHORT_SHA`, …
  - cf. https://docs.gitlab.com/ee/ci/variables/predefined_variables.html

GitLab CI/CD – Triggers

https://gitlab.com



`only` / `except`

- a pipeline is usually started after each push
- jobs can be restricted to or excluded from specific triggers
- `only` or `except` can be used to restrict to or excluded from:
  - `only:refs` or `except:refs` → branches/tags
  - `only:variables` or `except:variables` → values of variables
  - `only:changes` or `except:changes` → specific changes
- execution of pipelines can also be scheduled at specific times

```yaml
analyze_coverage:
  stage: test
  only:
    refs:
      - main            # branch name
      - tags            # all tags
      - merge_requests  # all MR
    changes:
      - **/*.cs
  except:
    variables:
      - $CI_COMMIT_MESSAGE =~ /skip-coverage/
```

```yaml
job1:
  stage: build
  only:
    refs:
      - main

job2:
  stage: build
  only:          # defaults to refs
    - main       # -> job1 and job2 are equivalent
```

GitLab CI/CD – Artifacts

https://gitlab.com



artifacts

- artifacts are results of a job which should be stored on the GitLab server (e.g., binaries, reports, …)
- as jobs are isolated and independent of each other, artifacts can be used to pass results of one job to the next one (typical example `build` → `test` to reuse compiled binaries in test stage)
- by default, each jobs receives all artifacts from all previous jobs
- jobs can define dependencies to explicitly set which artifacts are required

```yaml
build:
  stage: build
  artifacts:
    name: $CI_COMMIT_SHORT_SHA  # filename of generated artifact
    expire_in: 1 week           # duration to store artifact on GitLab server
    paths:                      # list of paths to include in artifact
      - "**/bin/*"
      - "**/obj/*"
  script:
    - dotnet build

test:
  stage: test
  dependencies:                 # list of artifacts to include from previous jobs
    - build
  script:
    - dotnet test
```

GitLab CI/CD – Caches

https://gitlab.com



cache

- if the docker executor is used, each job is executed in a new container
- required dependencies are downloaded each time
- jobs can define paths which should be cached
- cached files are stored locally in the runner (default) or in a cloud-based cache (e.g., S3 buckets)
- `cache:key` can be used to create independent caches (e.g., **`key: "$CI_COMMIT_REF_SLUG"`** → cache per branch/tag)
- `cache:policy` defaults to `pull-push` and can be set to `pull`, if cache is not changed

```yaml
image: mcr.microsoft.com/dotnet/sdk:6.0

variables:
  nuget_packages_cache: ".nuget"

build:
  stage: build
  cache:
    paths:                      # list of paths to include in cache
      - $nuget_packages_cache
  script:
    - dotnet restore --packages $nuget_packages_cache
    - dotnet build --no-restore
```

GitLab CI/CD – Environments

https://gitlab.com

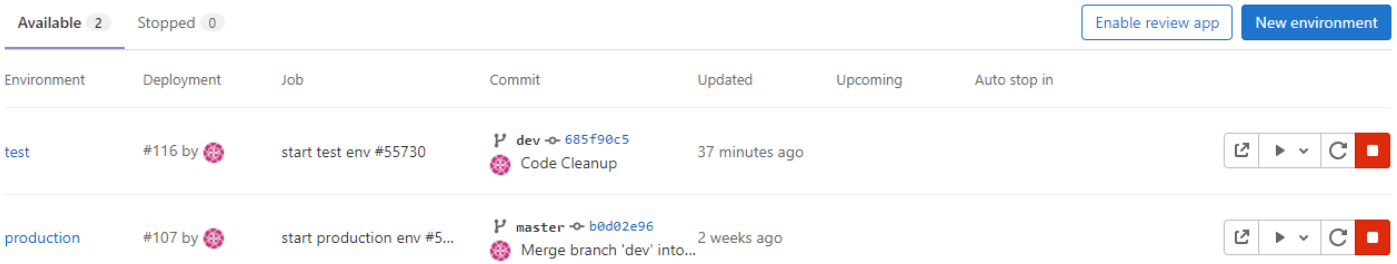


environment

- jobs can create environments to test, review, or operate an application
- environments can be accessed in the web frontend (Operations → Environments)
- specific jobs can be defined to stop a deployed environment again
- environments can be created manually or automatically

```yaml
deploy_test_environment:
  stage: deploy
  dependencies:
    - build
  environment:
    name: test
    action: start
    url: https://my.test.environment.com  # URL of the deployed environment
    on_stop: "stop_test_environment"
  when: manual                            # deployment is triggered manually
  script:
    - ...                                 # commands to deploy test environment
```

```yaml
stop_test_environment:
  stage: deploy
  environment:
    name: test
    action: stop
  script:
    - ...                 # commands to stop test environment
```



GitHub Actions

https://github.com/features/actions



- GitHub offers a VM-based CI/CD platform
  - definition of CI/CD pipeline is stored in the repository (`.github/workflows/my_workflow.yml`)
  - available operating systems of runners are Windows Server, Ubuntu Linux, macOS
  - each pipeline is executed on a fresh Azure VM (Dv2- and DSv2-series)
    - 2 cores, 7GB memory, 14GB SSD disk space
  - macOS machines are hosted directly by GitHub
    - 3 cores, 14GB memory, 14GB SSD disk space
  - maximum runtime of a job on a GitHub hosted runner is 6 hours
  - runners can also be self-hosted

- Actions are building blocks of pipelines
  - many actions available which are developed and maintained by the community
  - execution of shell commands is also supported

- Online documentation of GitHub Actions: https://docs.github.com/en/actions

GitHub Actions

https://github.com/features/actions

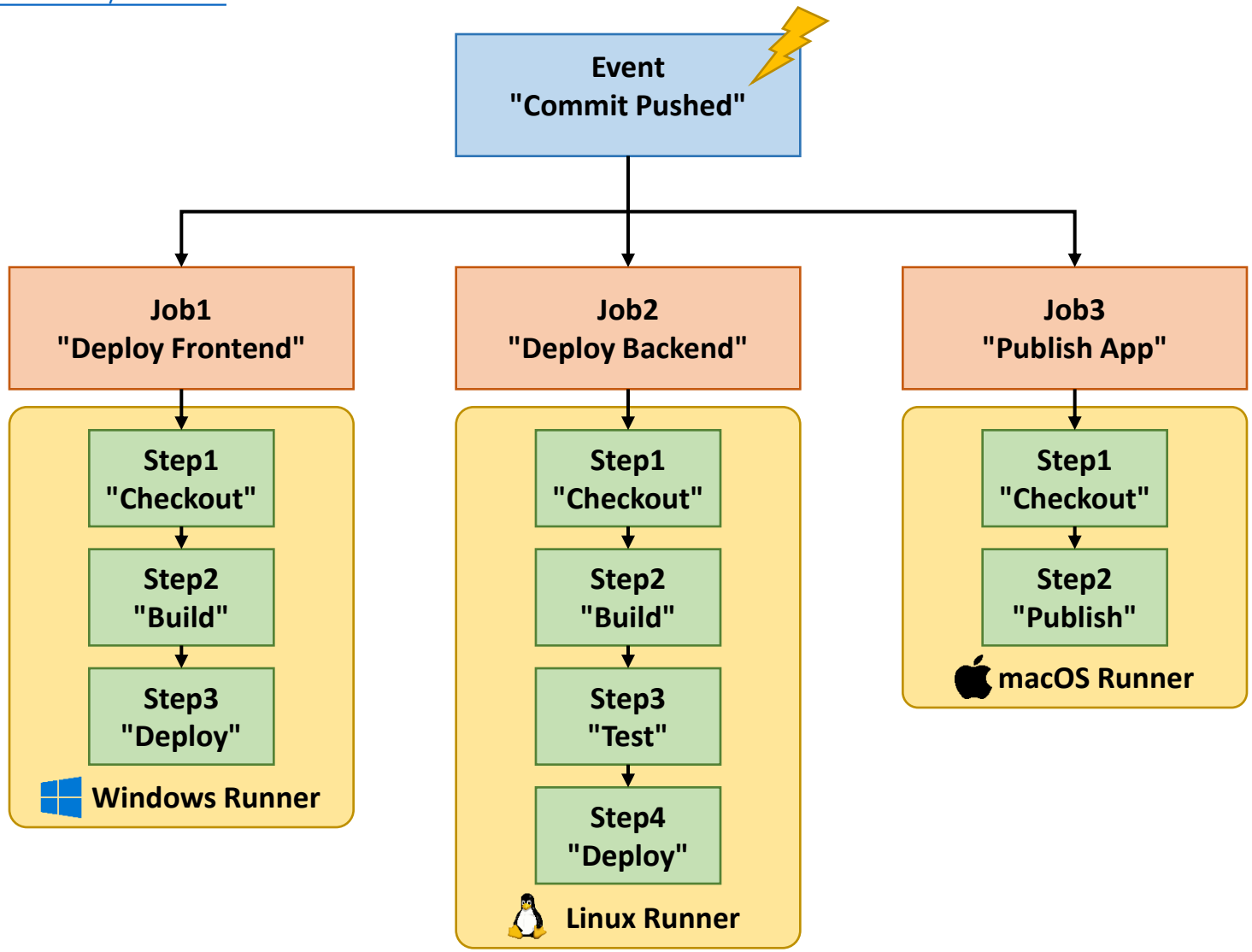


- Event
  - activity that triggers a workflow
  - e.g., push to repository, create a PR, …
- Workflow
  - a number of jobs
  - scheduled or triggered by an event
  - jobs executed in parallel per default
- Job
  - a series of steps
  - executed by a runner
- Step
  - action or shell command
  - all steps within a job executed by the same runner
- Action
  - standalone commands
  - lots of community actions
  - e.g., checkout repository, coverage tests, installation of software, …
- Runner
  - client that executes a job
  - self-hosted or hosted by GitHub
  - Windows, Linux, macOS images

GitHub Actions

https://github.com/features/actions

GitHub Actions



GitHub Actions

https://github.com/features/actions



GitHub Actions

- Configuration is stored in the repository
  - actions are defined in a YAML file (`*.yml`)
  - must be committed to `.github/workflows`
- Simple example: `learn-github-actions.yml`

```yaml
name: learn-github-actions           # name of the workflow, optional
on: [push]                           # event that triggers the workflow
jobs:
  check-bats-version:                # id of the first job
    runs-on: ubuntu-latest           # specifies the runner to use
                                     # > provisions fresh container hosted by github
    steps:
      - uses: actions/checkout@v3    # step 1: executes community action "checkout@v3"
                                     # > checks out repository
      - uses: actions/setup-node@v1  # step 2: executes community action "setup-node@v1"
                                     # > sets up node environment, provides npm command
      - run: npm install -g bats     # step 3: shell command
                                     # > installs node.js package "bats"
      - run: bats -v                 # step 4: shell command
                                     # > executes "bats -v"
```

GitHub Actions – Triggers

https://github.com/features/actions



on

- single GitHub event or list of events that trigger the workflow

```yaml
on: [push, pull_request] # events that trigger the workflow
```

- further trigger behavior specification possible

```yaml
on:
  # triggers workflow when pushing to branch 'main' or tags 'v1.0' .. 'v1.9'
  # triggers workflow when a PR for branch 'main' is created
  push:
    branches:
      - main
    tags:
      - v1.*    # wildcard support
  pull_request:
    branches:
      - main
```

GitHub Actions – Triggers

https://github.com/features/actions



- workflows can be scheduled using the schedule event

```yaml
on:
  schedule:
    # POSIX cron syntax
    # when using *, string needs to be quoted
    - cron: '30 5,17 * * *' # triggers at 05:30 and 17:30
```

- workflows can also be triggered manually (on GitHub actions page)

```yaml
on: workflow_dispatch
```

GitHub Actions – Jobs

https://github.com/features/actions



GitHub Actions

jobs

- lists the jobs that define the workflow

```yaml
jobs:
  my_first_job:         # id of first job
    name: My first job
  my_second_job:        # id of second job
    name: My second job
```

- all jobs within a workflow are executed in parallel per default
- a job is always executed by a specific runner

GitHub Actions – Jobs

https://github.com/features/actions



`jobs.<job_id>.runs-on`

- specifies the runner that should execute the job

```yaml
runs-on: ubuntu-latest
```

- GitHub provides various runners to choose from
  - `windows-latest` → Windows Server 2019
  - `ubuntu-latest` → Ubuntu 20.04
  - `macOS-latest` → macOS Catalina 10.15
  - …
- self-hosted runners can also be used
  - runners can be added in the repository settings

```yaml
runs-on: [self-hosted, linux]
```

GitHub Actions – Jobs

https://github.com/features/actions



GitHub Actions

jobs.<job_id>.needs

- allows to specify which jobs need to complete first

```yaml
jobs:
  job1:
  job2: # will be executed when 'job1' succeeds
    needs: job1
  job3: # will be executed when 'job1' and 'job2' succeed
    needs: [job1, job2]
```

- conditional expressions can be used to determine job execution

```yaml
jobs:
  job1:
  job2:
    needs: job1
  job3:
    # will always be executed after 'job1' and 'job2',
    # regardless of whether or not they succeed
    if: always()
    needs: [job1, job2]
```

GitHub Actions – Steps

https://github.com/features/actions



jobs.<job_id>.steps

- specifies the steps of a job

```yaml
jobs:
  job:
    steps:
      - name: Step 1
      - name: Step 2
```

- used to execute shell commands or run actions

GitHub Actions – Steps

https://github.com/features/actions



GitHub Actions

jobs.<job_id>.steps[*].run

- executes a shell command

```yaml
jobs:
  job:
    steps:
      - name: Build Project
        run: dotnet build
```

```yaml
jobs:
  job:
    steps:
      - name: Step 1
        run: | # multi-line command
          echo "GitHub"
          echo "Actions"
```

- working directory can also be set

```yaml
jobs:
  job:
    steps:
      - name: Build Project
        run: dotnet build
        working-directory: ./src
```

GitHub Actions – Actions

https://github.com/features/actions



jobs.<job_id>.steps[*].uses

- executes a GitHub action (i.e., a reusable unit of code)

```yaml
steps:
  - uses: actions/checkout@v3
```

- many community actions available
- specific version to be used can be defined via
  - tag: actions/foo-bar@v1.0.0
  - branch name: actions/foo-bar@main
  - commit SHA: actions/foo-bar@06911ba...

GitHub Actions – Actions

https://github.com/features/actions



GitHub Actions

jobs.<job_id>.steps[*].with

- specifies input parameters defined by an action

```yaml
jobs:
  job:
    steps:
    - name: Setup dotnet
      uses: actions/setup-dotnet@v2
      with:
        dotnet-version: '6.0.x'
        include-prerelease: true
```

GitHub Actions – Strategies

https://github.com/features/actions



jobs.<job_id>.strategy

- used to create different job variations using a build matrix

jobs.<job_id>.strategy.matrix

- defines a matrix for various job configurations
- a maximum of $2^8 = 256$ variations can be created per workflow run

GitHub Actions – Strategies

https://github.com/features/actions



GitHub Actions

- this example will create 6 jobs which might all run in parallel

```yaml
jobs:
  build:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-18.04, ubuntu-20.04]
        dotnet: [3.1.x, 5.0.x, 6.0.x]
    name: build on ${{ matrix.os }} using dotnet ${{ matrix.dotnet }}
    steps:
      - uses: actions/checkout@v3
      - name: Setup dotnet
        uses: actions/setup-dotnet@v2
        with:
          dotnet-version: ${{ matrix.dotnet }}
      - run: dotnet build
```

jobs.<job_id>.strategy.max-parallel

- limits parallel execution

```yaml
strategy:
  max-parallel: 2
```

GitHub Actions – Env. Vars

https://github.com/features/actions



env

- a map of environmental variables available to all jobs within the workflow

```yaml
env:
  VARIABLE1: value1
  VARIABLE2: value2
```

- can also be specified only for
  - jobs: **`jobs.<job_id>.env`**
  - steps: **`jobs.<job_id>.steps[*].env`**

```yaml
jobs:
  job:
    env:
      FOO: bar
    steps:
      - name: Step 1
        env:
          TEST: "GitHub Actions"
        run: |
          echo "FOO = $FOO"
          echo "TEST = $TEST"
```

GitHub Actions – Env. Vars

https://github.com/features/actions



- do **not** store secrets in workflow configuration files!
- use GitHub secrets to set values of environmental variables

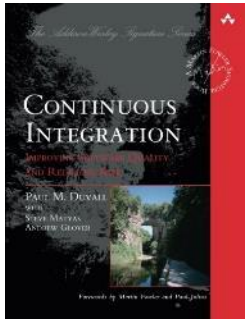
```yaml
jobs:
  job1:
    env:
      SUPER_SECRET: ${{ secrets.SUPER_SECRET }}
```

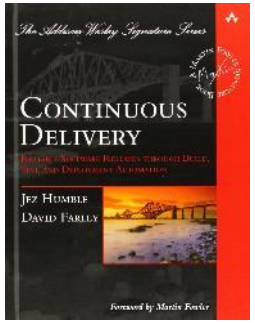
- secrets can be created in the repository settings

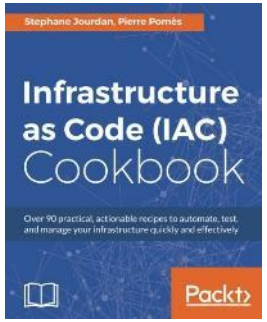
Automatically Provide IT Infrastructure

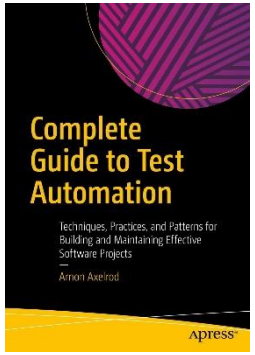
- Continuous delivery/deployment requires additional tools to provide test or even production environments automatically
- Configuration management (CM) tools manage and automate the configuration of IT infrastructure (e.g., virtual machines, databases, containers)
  - examples for CM tools: Puppet, Chef, Ansible
- Infrastructure as Code (IaC)
  - required infrastructure is defined in configuration files which are managed like source code (e.g., version controlled)
  - foundation for automating infrastructure provisioning

Literature









P. Duvall:
Continuous Integration: Improving Software Quality and Reducing Risk
Addison Wesley

J. Humble, D. Farley:
Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation
Addison Wesley

S. Jourdan, P. Pomes:
Infrastructure as Code (IAC) Cookbook
Packt Publishing

A. Axelrod:
Complete Guide to Test Automation: Techniques, Practices, and Patterns for Building and Maintaining Effective Software Projects
Apress